

theory__to__code

理論式 ↔ コード 対応ガイド

「数式がプログラムのどこにあるか」を 1 対 1 で対応させた解説。

構成マップ

物理の問題（熱拡散 PDE）	
↓ 有限差分で空間離散化	
連続時間 ODE: $\dot{\theta} = A_c \theta + B_c u$	← thermal_model.py
↓ 前進オイラー法で時間離散化	
離散時間方程式: $\theta_{k+1} = A_d \theta_k + B_d u_k$	← thermal_model.py
↓ ノイズを加えてカルマンフィルタに渡す	
カルマンフィルタ: 予測 → 更新	← kalman_filter.py
↓ 推定温度から変位を計算	
熱変位: $\delta = \alpha \sum (\theta_i - T_{ref}) \Delta x$	← main.py

PART 1: 熱拡散方程式 → 状態行列 A_c

理論式

1 次元熱拡散方程式（10 節点モデルでは内部節点 $i = 1, 2, \dots, 8$ ）:

$$\rho c_p \frac{d\theta_i}{dt} = \frac{k}{(\Delta x)^2} (\theta_{i-1} - 2\theta_i + \theta_{i+1})$$

両辺を ρc_p で割り、熱拡散率 $\alpha = k/(\rho c_p)$ を使うと:

$$\frac{d\theta_i}{dt} = \frac{\alpha}{(\Delta x)^2} (\theta_{i-1} - 2\theta_i + \theta_{i+1})$$

端部節点（節点 0）に対流冷却と熱入力を追加:

$$\frac{d\theta_0}{dt} = \frac{\alpha}{(\Delta x)^2}(\theta_1 - \theta_0) - \frac{h_{\text{conv}}}{\rho c_p \Delta x} \theta_0 + \frac{u}{\rho c_p \Delta x}$$

これを行列形式にすると $\dot{\boldsymbol{\theta}} = \mathbf{A}_c \boldsymbol{\theta} + \mathbf{B}_c u$ で：

$$\mathbf{A}_c = \begin{bmatrix} -r-h & r & 0 & 0 & 0 \\ r & -2r & r & 0 & 0 \\ 0 & r & -2r & r & 0 \\ 0 & 0 & r & -2r & r \\ 0 & 0 & 0 & r & -r-h \end{bmatrix}, \quad \mathbf{B}_c = \begin{bmatrix} 1/(\rho c_p \Delta x) \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

ここで $r = \alpha/(\Delta x)^2$, $h = h_{\text{conv}}/(\rho c_p \Delta x)$

対応するコード (thermal_model.py)

```
def _build_continuous(self):
    r = self.alpha / self.dx ** 2          # ← α/(Δ x)^2 [1/s]
    h = self.h_conv / (self.rho_cp * self.dx) # ← h_conv/(ρ cp · Δ x) [1/s]

    A = np.zeros((n, n))

    # 内部節点 i=1..8: dθ_i/dt = r(θ_{i-1} - 2θ_i + θ_{i+1})
    for i in range(1, n - 1):
        A[i, i-1] = r      # ← 係数 +r (左隣から)
        A[i, i] = -2*r     # ← 係数 -2r (自分)
        A[i, i+1] = r      # ← 係数 +r (右隣から)

    # 端部節点 (対流項 -h が加わる)
    A[0, 0] = -r - h; A[0, 1] = r # ← -r-h (境界)
    A[-1, -1] = -r - h; A[-1, -2] = r

    B = np.zeros((n, 1))
    B[0, 0] = 1.0 / (self.rho_cp * self.dx) # ← 1/(ρ cp · Δ x)
```

PART 2：前進オイラー法による離散化 → A_d, B_d

理論式

連続時間方程式 $\dot{\boldsymbol{\theta}} = \mathbf{A}_c \boldsymbol{\theta} + \mathbf{B}_c u$ を

タイムステップ Δt ごとの差分方程式に変換する。

考え方：微分を差分で近似する

$$\frac{d\boldsymbol{\theta}}{dt} \approx \frac{\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k}{\Delta t}$$

これを代入して $\boldsymbol{\theta}_{k+1}$ について解くと：

$$\theta_{k+1} = \underbrace{(\mathbf{I} + \mathbf{A}_c \Delta t)}_{\mathbf{A}_d} \theta_k + \underbrace{\mathbf{B}_c \Delta t}_{\mathbf{B}_d} u_k$$

式の意味：「現在の温度 θ_k から、 Δt 秒間だけ変化率 $\dot{\theta}$ が一定だと仮定して次の温度を計算する」。

$$\boxed{\mathbf{A}_d = \mathbf{I} + \mathbf{A}_c \Delta t, \quad \mathbf{B}_d = \mathbf{B}_c \Delta t}$$

対応するコード (thermal_model.py)

```
def _discretize(self):
    # A_d = I + A_c · Δ t
    A_d = np.eye(self.n) + self.A_c * self.dt

    # B_d = B_c · Δ t
    B_d = self.B_c * self.dt

    return A_d, B_d
```

scipy の expm は不要になり、numpy だけで計算できる。

安定条件

前進オイラー法は Δt が大きすぎると数值的に発散する可能性がある。

安定条件： $r \cdot \Delta t \leq 0.5$ ($r = \alpha / (\Delta x)^2$)

このシミュレーションでは：

$$r \cdot \Delta t = \frac{1.2 \times 10^{-5}}{0.1^2} \times 10 = 1.2 \times 10^{-3} \times 10 = 0.012 \ll 0.5 \quad \checkmark$$

$\Delta t = 10\text{s}$ 、 $\Delta x = 0.1\text{m}$ の条件では安定であることが確認できる。

PART 3：カルマンフィルタ 予測ステップ

理論式（予測）

$$\hat{\theta}_k^- = \mathbf{A}_d \hat{\theta}_{k-1} + \mathbf{B}_d u_{k-1}$$

$$\mathbf{P}_k^- = \mathbf{A}_d \mathbf{P}_{k-1} \mathbf{A}_d^\top + \mathbf{Q}$$

- $\hat{\theta}_k^-$ ：センサ情報を使う前の「モデルだけによる予測（事前推定）」
- $\mathbf{P}_k^-$ ：予測の誤差共分散（モデル予測で不確かさは増える $\rightarrow +\mathbf{Q}$ ）
- $\mathbf{Q}$ ：プロセスノイズ共分散（モデル誤差の大きさを表す）

対応するコード (kalman_filter.py)

```
def predict(self, u):
    #  $\hat{\theta}_k^- = A_d \hat{\theta}_{k-1} + B_d u_{k-1}$ 
    self.x = self.A @ self.x + self.B @ u

    #  $P_k^- = A_d P_{k-1} A_d^T + Q$ 
    self.P = self.A @ self.P @ self.A.T + self.Q
```

PART 4：カルマンフィルタ 更新ステップ

理論式 (更新)

イノベーション共分散 (予測と観測の合計不確かさ)：

$$S_k = H P_k^- H^T + R$$

カルマンゲイン (センサ vs モデルの信頼度の比)：

$$K_k = P_k^- H^T S_k^{-1}$$

状態更新 (イノベーション $\nu_k = y_k - H \hat{\theta}_k^-$ で補正)：

$$\hat{\theta}_k = \hat{\theta}_k^- + K_k \nu_k$$

共分散更新：

$$P_k = (I - K_k H) P_k^-$$

対応するコード (kalman_filter.py)

```
def update(self, y):
    #  $S = H P_k^- H^T + R$  ← イノベーション共分散
    S = self.H @ self.P @ self.H.T + self.R

    #  $K = P_k^- H^T S^{-1}$  ← カルマンゲイン
    K = self.P @ self.H.T @ np.linalg.inv(S)

    #  $\nu = y - H \hat{\theta}_k^-$  ← イノベーション (予測と計測の差)
    innov = y - self.H @ self.x

    #  $\hat{\theta}_k = \hat{\theta}_k^- + K \nu$  ← センサ情報で補正
```

```

self.x = self.x + K @ innov

# P = (I - KH) P^- ← 共分散更新（不確かさが減る）
self.P = (np.eye(self.x.shape[0]) - K @ self.H) @ self.P

```

PART 5：カルマンゲイン K の物理的意味

理論：スカラー版で直感を掴む

スカラー（1 節点・1 センサ）の場合に簡略化すると：

$$K = \frac{P^-}{P^- + R}$$

状況	P^- vs R	K の値	意味
モデルが不確か	$P^- \gg R$	$K \approx 1$	センサをほぼ信頼
センサがノイズだらけ	$P^- \ll R$	$K \approx 0$	モデルをほぼ信頼
両方同じ	$P^- = R$	$K = 0.5$	半々で信頼

更新式 $\hat{\theta} = \hat{\theta}^- + K(\underbrace{y - H\hat{\theta}^-}_{\text{予測との差}})$ は：

- イノベーション（予測との差）が大きいほど大きく補正
- K が大きいほどセンサに引っ張られる
- K が小さいほどモデル予測を維持する

コードで確認できる場所

```

# kalman_filter.py の update メソッド内
S = self.H @ self.P @ self.H.T + self.R # P^- + R に対応
K = self.P @ self.H.T @ np.linalg.inv(S) # P^- / (P^- + R) に対応

# main.py の初期設定
Q_mat = np.eye(N_NODES) * PROC_NOISE_STD ** 2 # = 0.03^2 = 0.0009
R_mat = np.eye(n_obs) * OBS_NOISE_STD ** 2 # = 0.20^2 = 0.04
# → R >> Q なので「センサよりモデルをやや信頼」という設定

```

PART 5-A：カルマンゲイン K はなぜこの式になるか（MMSE 導出）

問い：なぜ $K = P^- H^\top (H P^- H^\top + R)^{-1}$ なのか

カルマンゲインは「事後誤差共分散 P_k のトレース（対角和）を最小化する K 」として導ける。

つまり「全節点の推定誤差の 2 乗和を最小にする最適な重み」である。

導出

更新後の状態 $\hat{\theta} = \hat{\theta}^- + K(y - H\hat{\theta}^-)$ の真値との誤差を e とする：

$$e = \theta - \hat{\theta} = \theta - \hat{\theta}^- - K(y - H\hat{\theta}^-)$$

$y = H\theta + v$ (v ：観測ノイズ) を代入して：

$$e = \theta - \hat{\theta}^- - K(H\theta + v - H\hat{\theta}^-) = (I - KH) \underbrace{(\theta - \hat{\theta}^-)}_{e^-} - Kv$$

事後誤差共分散 $P = \mathbb{E}[ee^\top]$ を計算する (e^- と v は独立)：

$$P = (I - KH)P^-(I - KH)^\top + KRK^\top$$

これを展開すると：

$$P = P^- - KHP^- - P^-H^\top K^\top + K(HP^-H^\top + R)K^\top$$

$\text{tr}(P)$ を最小化するために K で微分して $= 0$ とおく：

$$\frac{\partial \text{tr}(P)}{\partial K} = -2(P^-H^\top)^\top + 2K(HP^-H^\top + R) = 0$$

$$\therefore \boxed{K = P^-H^\top \underbrace{(HP^-H^\top + R)^{-1}}_{S^{-1}}}$$

導出の読み方

式の中の項	意味
$P^-H^\top$	「モデル予測の不確かさ」×「センサが何を見ているか」＝ 予測とセンサのクロス共分散
$HP^-H^\top$	センサ空間に投影した予測の不確かさ
$HP^-H^\top + R$	センサ空間でのトータルの不確かさ（モデル誤差＋センサノイズ）
$(HP^-H^\top + R)^{-1}$	不確かさで正規化（大きい不確かさほど割り引く）

直感的に言えば：

$$K = \frac{\text{予測の不確かさ (センサ空間)}}{\text{予測の不確かさ} + \text{センサのノイズ}}$$

PART 5-B：カルマンゲインの数値例（このシミュレーションの実際の値）

初期（t = 0）の K の値

このシミュレーションの設定：

$P_0 = 2.0 \times I_{1_0}$ (初期不確かさ：大きめに設定)
 $Q = 0.03^2 \times I_{1_0} = 0.0009 \times I_{1_0}$
 $R = 0.20^2 \times I_2 = 0.04 \times I_2$
 $H = \begin{bmatrix} 1, 0, 0, 0, 0, 0, 0, 0, 0, 0 \\ 0, 0, 0, 0, 0, 0, 0, 0, 0, 1 \end{bmatrix}$, (節点 0・9 をセンサとして観測)

スカラー近似で初期 K を計算すると：

$$K_{\text{init}} \approx \frac{P_0}{P_0 + R} = \frac{2.0}{2.0 + 0.04} \approx 0.98$$

→ 初期はほぼ全面的にセンサを信頼（ $K \approx 1$ ）。

初期の P が大きい（「最初は自信なし」）ため、センサ計測を強く採用する。

定常状態（t » 20min）の K の値

KF が収束した後、P は定常値 P_∞ に落ち着く。この P は代数リッカチ方程式の解：

$$P_\infty = A_d P_\infty A_d^\top + Q - A_d P_\infty H^\top (H P_\infty H^\top + R)^{-1} H P_\infty A_d^\top$$

定常では $P_\infty \ll P_0$ となり（FEM モデルが蓄積したセンサ情報で精度向上）、

定常カルマンゲイン K_∞ は小さくなる：

$$K_\infty \approx \frac{P_\infty}{P_\infty + R}$$

$P_\infty \approx 0.01$ （定常値）とすると：

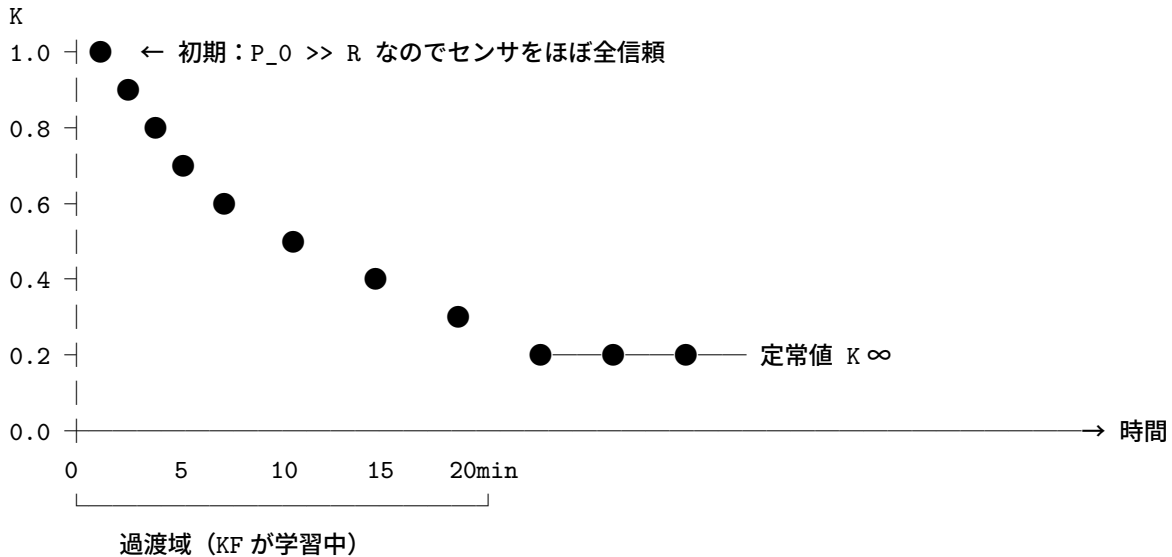
$$K_\infty \approx \frac{0.01}{0.01 + 0.04} = 0.2$$

→ 定常では「センサ 2 割・モデル 8 割」程度の信頼比率。

モデルが十分に状態を追えているので、センサへの依存度が下がる。

K の時間変化のイメージ

カルマンゲイン K の推移



この収束は kf_sigma (推定標準偏差 σ) のグラフ (パネル 7) と対応している。

PART 5-C：イノベーション (予測残差) の意味

定義

$$\nu_k = y_k - \mathbf{H}\hat{\theta}_k^-$$

- y_k : センサが実際に計測した値
- $\mathbf{H}\hat{\theta}_k^-$: 「モデルだけで予測したらセンサはこの値のはず」という予測

つまり「センサ計測とモデル予測のズレ」。これをイノベーション (新情報) という。

イノベーションが果たす役割

イノベーション ν が大きい場合:

- モデルがセンサと大きくズレている
- モデルに知らない現象が起きている (工作機械の突発的な熱発生など)
- $K \times \nu$ が大きく、状態推定を大きく修正する

イノベーション ν がほぼゼロの場合:

- モデルの予測がセンサとよく合っている
- FEM モデルが現象を正確に追えている
- 修正量が小さく、推定値はモデル予測をほぼ維持する

コードでの対応

```
# kalman_filter.py の update
innov = y - self.H @ self.x      # ← v = y_k - H θ̂_k^-
self.x = self.x + K @ innov      # ← θ̂ = θ̂^- + K v
```

イノベーション v の統計的性質（健全性チェック）:

KF が正しく動作しているとき、 v はホワイトノイズ（系列無相関）になるはず。

v に自己相関が残る場合はモデルに系統誤差があるサイン。

PART 5-D：更新後の共分散 P が「なぜ小さくなるか」

式の意味

$$P_k = (I - K_k H) P_k^-$$

$(I - KH)$ は P^- に対する「縮小係数」として働く。

スカラー版で確認：

$$P = \left(1 - \frac{P^-}{P^- + R}\right) P^- = \frac{R}{P^- + R} \cdot P^- = \frac{P^- R}{P^- + R}$$

これは「 P^- と R の調和平均」に相当し、必ず $P < P^-$ かつ $P < R$ となる。

更新前 P^-	センサノイズ R	更新後 P
2.0 °C ² (初期)	0.04 °C ²	$2.0 \times 0.04 / 2.04 \approx 0.039$ °C ²
0.01 °C ² (定常)	0.04 °C ²	$0.01 \times 0.04 / 0.05 = 0.008$ °C ²

→ センサが1回届くだけで、不確かさが大幅に減る (2.0 → 0.039)。

コードとの対応

```
# kalman_filter.py の update
self.P = (np.eye(self.x.shape[0]) - K @ self.H) @ self.P
#           ↑ (I - KH) という縮小係数           ↑ P^- に掛ける
```

パネル (7) 「推定不確かさの収束」グラフはこの P の対角成分の平方根

$\sigma_i = \sqrt{P_{ii}}$ を時系列でプロットしたもの。

PART 6：観測行列 $\mathbf{H}$ の設計

理論：観測方程式

$$\mathbf{y}_k = \mathbf{H}\boldsymbol{\theta}_k + \mathbf{v}_k, \quad \mathbf{v}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{R})$$

$\mathbf{H}$ は「どの節点をセンサで計測するか」を規定する行列。

Case A（温度センサ 節点 0・9）の $\mathbf{H}$ ：

$$\mathbf{H}_A = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \Rightarrow \mathbf{y} = \begin{bmatrix} \theta_0 \\ \theta_9 \end{bmatrix}$$

Case B（温度 2 点 + 変位センサ）の $\mathbf{H}$ ：

$$\mathbf{H}_B = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ c & c & c & c & c & c & c & c & c & c \end{bmatrix} \Rightarrow \mathbf{y} = \begin{bmatrix} \theta_0 \\ \theta_9 \\ c \sum_{i=0}^9 \theta_i \end{bmatrix}$$

ここで $c = \alpha_{\text{exp}} \cdot \Delta x \cdot 10^6 = 0.12 \mu\text{m}/^\circ\text{C}$

対応するコード (main_disp_obs.py)

```
# C_DISP = 12e-6 * 0.1 * 1e6 = 0.12 μm/°C/node
C_DISP = ALPHA_EXP * DX * 1e6

# 変位観測行（変位 = 全節点温度の加重和）
H_disp = C_DISP * np.ones((1, N_NODES)) # ← 10 節点すべてに同じ係数を掛ける

# Case B: 温度行 + 変位行を縦に積む
H_B = np.vstack([H_temp, H_disp])
```

観測可能性 (Observability) との関係：

$\text{rank}(\mathbf{O}) = n$ が状態推定の条件 ($\mathbf{O} = [\mathbf{H}^\top, (\mathbf{H}\mathbf{A}_d)^\top, \dots]^\top$)。

$\mathbf{H}$ に変位行を追加すると観測可能性が向上し、推定精度が上がる。

PART 7：熱変位の計算

理論式

線熱膨張の公式から棒全体の変位：

$$\delta_k = \alpha_{\text{exp}} \sum_{i=0}^{n-1} (\theta_{i,k} - \theta_{\text{ref}}) \cdot \Delta x \quad [\text{m}]$$

これを μm に変換するために 10^6 を掛ける：

$$\delta_k[\mu\text{m}] = \alpha_{\text{exp}} \cdot \Delta x \cdot 10^6 \cdot \sum_{i=0}^9 (\theta_{i,k} - 20)$$

対応するコード (main.py)

```
def compute_displacement(temps, t_ref=T_INIT):
    delta_T = temps - t_ref                                # (θ_i - θ_ref) の行列
    return ALPHA_EXP * np.sum(delta_T, axis=1) * DX * 1e6
#         ↑ α_exp         ↑ 全節点の和         ↑ Δx   ↑ m → μm 変換
```

temps の形状は (N_STEPS, N_NODES) = (540, 10)。

axis=1 で節点方向に総和をとり、各タイムステップの変位 (540,) を得る。

PART 8 : RMSE の計算

理論式

推定誤差の評価指標 (Root Mean Squared Error) :

$$\text{RMSE}_i = \sqrt{\frac{1}{T_{\text{eval}}} \sum_{k=k_{\text{start}}}^{N_{\text{steps}}} (\theta_{i,k} - \hat{\theta}_{i,k})^2}$$

定常域 (後半 1/3) のみで計算することで、KF 収束前の過渡域の影響を除く。

対応するコード (main.py)

```
t_start = 2 * N_STEPS // 3          # 定常域の開始 (= 360 ステップ目 = 60min)

def _calc_rmse(a, b):
    return float(np.sqrt(np.mean((a - b) ** 2)))

# 各節点の温度 RMSE
rmse_T = [_calc_rmse(res["true_T"][t_start:, i], res["kf_T"][t_start:, i])
          for i in range(N_NODES)]

# 変位 RMSE
rmse_disp = _calc_rmse(res["disp_true"][t_start:], res["disp_kf"][t_start:])
```

まとめ：理論式とコード行の対応一覧

理論式	ファイル	メソッド / 行
$r = \alpha/(\Delta x)^2$	thermal_model.py	_build_continuous L.67
$\mathbf{A}_c$ の三重対角	thermal_model.py	_build_continuous L.71-78
$\mathbf{B}_c[0] = 1/(\rho c_p \Delta x)$	thermal_model.py	_build_continuous L.82
$A_d = I + A_c \Delta t$ (前進オイラー)	thermal_model.py	_discretize L.100 np.eye(n) + A_c * dt
$\hat{\theta}_k^- = A_d \hat{\theta} + B_d u$	kalman_filter.py	predict L.67
$P_k^- = A_d P A_d^\top + Q$	kalman_filter.py	predict L.68
$S = H P^- H^\top + R$	kalman_filter.py	update L.79
$K = P^- H^\top S^{-1}$	kalman_filter.py	update L.80
$\hat{\theta} = \hat{\theta}^- + K \nu$	kalman_filter.py	update L.82
$P = (I - K H) P^-$	kalman_filter.py	update L.83
$\delta = \alpha \sum (\theta_i - T_{\text{ref}}) \Delta x$	main.py	compute_displacement L.113
$H_{\text{disp}} = c_{\text{disp}}[1, \dots, 1]$	main_disp_obs.py	build_H_and_R L.117